Kernal

Lecture# 4

Introduction

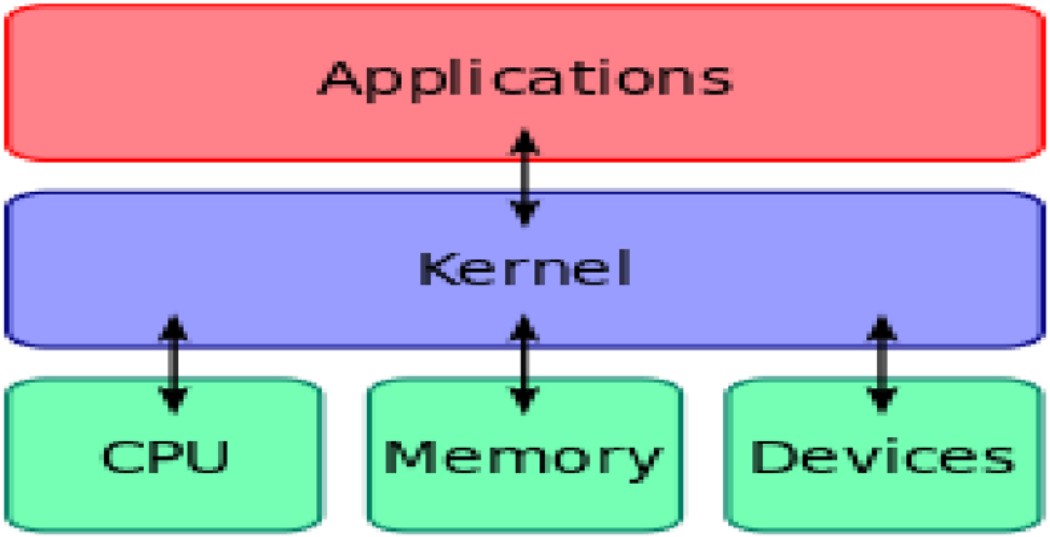
- A kernel is a central component of an operating system that acts as an interface between the user applications and the hardware.
- The sole aim of the kernel is to manage the communication between the software (user level applications) and the hardware (CPU, disk memory etc.)
- The main tasks of the kernel are :
  - Process management,
  - Device management,
  - Memory management,
  - Interrupt handling,
  - I/O communication,
  - File system management
  - Security and Access Control

Introduction

- As a basic component of an operating system, a kernel provides the lowest-level abstraction layer for the resources(especially memory, processors and I/O devices) that application must control to perform their function.
- It typically make these facilities available to application processes through inter-process communication mechanisms and system calls.

Introduction

- Kernel connects the application software to the hardware of a computer.



Features Provided by Kernel

- Scheduling of Process
- Interprocess Communication
- Process Synchronization
- Context Switching
- Manipulation of PCB
- Interrupt Handling
- Process Creation and Termination
- Process Suspension and Resumption

Types of Kernel

- ***Monolithic kernel:***

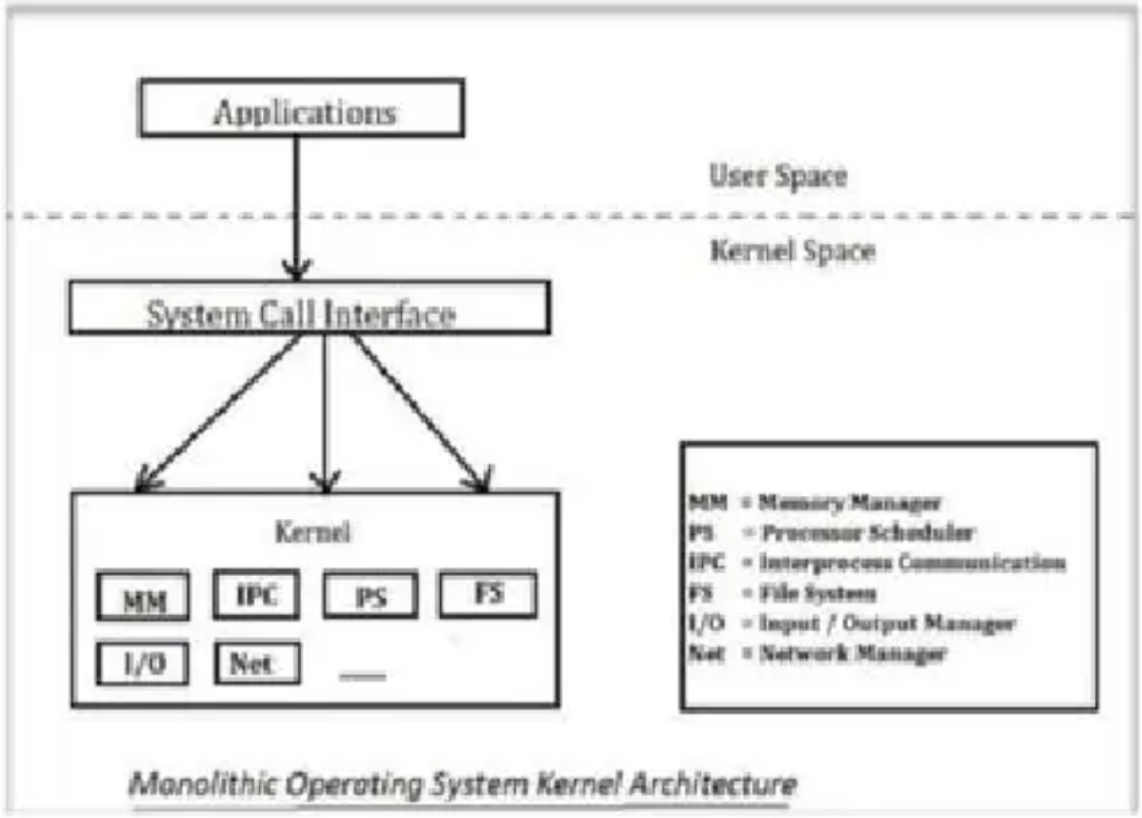
  A Monolithic kernel is a single large processes running entirely in a single address space.

  It runs every basic system services like process and memory management, Interrupt handling and I/O communications, File systems etc.

  Every component of the operating system is contained in the kernel and can directly communicate with each other simply by using function calls. The kernel typically executes with unrestricted access to the computer system.

  The examples of monolithic kernel based OSs are Linux, Unix

Types of Kernel



Types of Kernel

Advantages of Monolithic kernel:

1. Smaller in Source and Compiled forms
2. Less code generally means fewer bugs and security problems are also less.
3. System calls are used to do operations in Monolithic kernel.
4. Execution is fast.
5. It has all the things in kernel itself so we don't need any extra mechanism for handling of I/O and process at the time of the application making.

Types of Kernel

Disadvantages of Monolithic kernel:

1. Coding in Kernel space is hard, since you cannot use common libraries.
2. Debugging is harder, rebooting the computer is often needed.
3. Bugs in one part of kernel produce strong side effects.
4. Kernel often become very huge and difficult to maintain.
5. Not-Portable:- It must be rewritten for each new architecture that the OS is to be used on.

Types of Kernel

- ***Microkernels:***

  In Microkernels, the kernel is broken down into separate processes, known as servers. Some of the servers run in kernel space and some run in user-space.

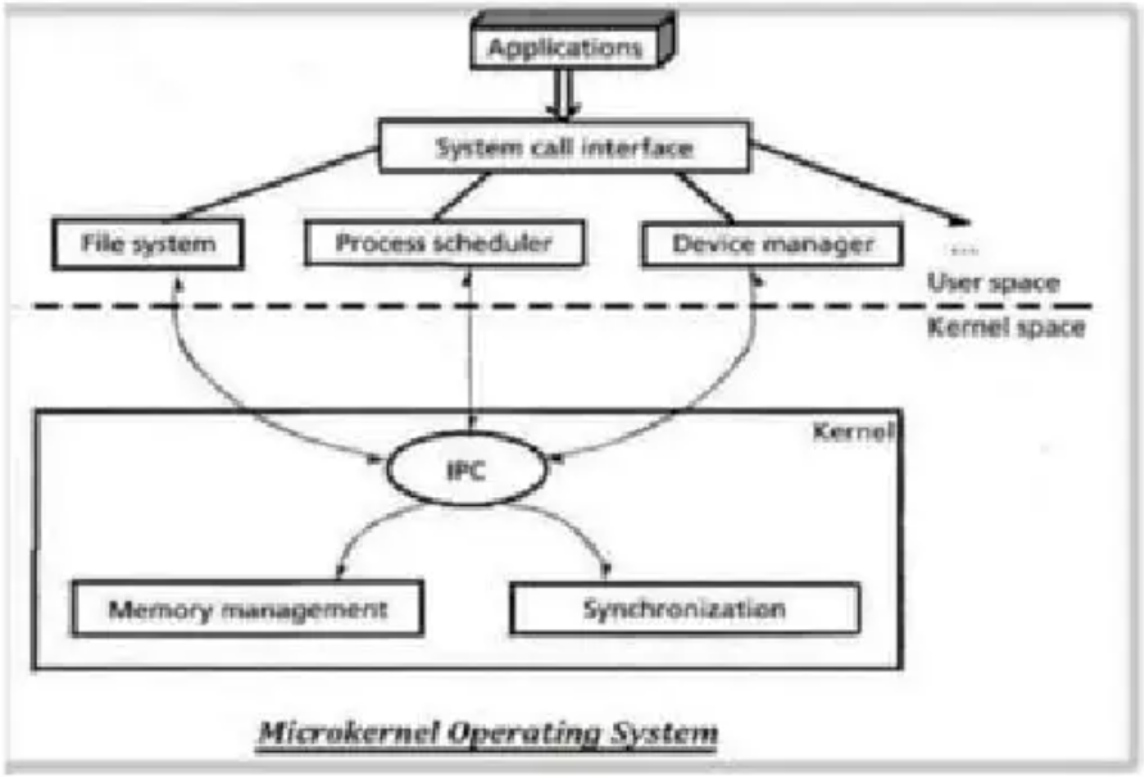
  All servers are kept separate and run in different address spaces.

  The communication in microkernels is done via message passing. The servers communicate through IPC (Interprocess Communication). Servers invoke "services" from each other by sending messages.

  The separation has advantage that if one server fails other server can still work efficiently.

  The example of microkernel based OS are Mac OS X and Windows NT.

Types of Kernel



Types of Kernel

Advantages of Microkernel.

1. Easier to maintain than Monolithic Kernel
2. Crash resistant (If one server fails, other servers can still work efficiently).
3. Portable
4. Smaller in size.
5. Contains smaller amount of code. It increases stability and security.

Types of Kernel

Disadvantages of Microkernel.

1. Slower Message Passing
2. More Complex
3. Limited Performance
4. High Memory Usage

Types of Kernel

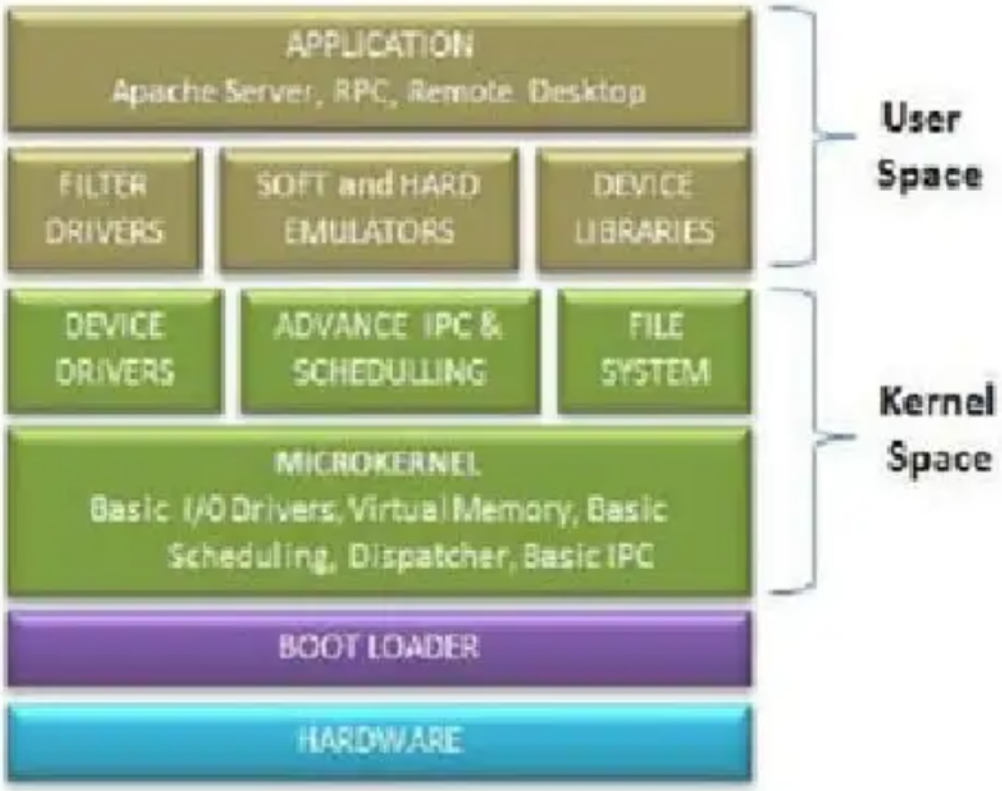
- ***Hybrid Kernels:***

  Hybrid kernel is a kernel architecture based on a combination of microkernel and monolithic kernel architecture used in computer operating systems.

  This kernel approach combines the speed and simpler design of monolithic kernel with the modularity and execution safety of microkernel.

  A hybrid kernel runs some services in the kernel space to reduce the performance overhead of a traditional microkernel, while still running kernel code as servers in the user space.

Types of Kernel



Types of Kernel

- ***Nanokernels:***

  A nanokernel is a very minimalist operating system kernel which represents the closest hardware abstraction layer of the operating system by interfacing the CPU, managing interrupts and interacting with the MMU.

  The interrupt management and MMU interface are not necessarily part of a nanokernel; however, on most architecture these components are directly connected to the CPU, therefore, it often makes sense to integrate these interfaces into the kernel. It lack system services

Types of Kernel

- ***Exokernels:***

  An exokernel is a type of kernel that does not abstract hardware into theoretical models. Instead it allocates physical hardware resources, such as processor time, memory pages, and disk blocks, to different programs.

Context Switching
(Kernel mode and User mode)

- ***Kernel Mode:***

  In Kernel mode, the executing code has complete and unrestricted access to the underlying hardware. It can execute any CPU instruction and reference any memory address. Kernel mode is generally reserved for the lowest-level, most trusted functions of the operating system. Crashes in kernel mode are catastrophic; they will halt the entire PC.

- ***User Mode:***

  In User mode, the executing code has no ability to directly access hardware or reference memory. Code running in user mode must delegate to system APIs to access hardware or memory. Due to the protection afforded by this sort of isolation, crashes in user mode are always recoverable. Most of the code running on your computer will execute in user mode.

Context Switching
(Kernel mode and User mode)

The following three situations result in switching to kernel mode from user mode of operation:

1. The scheduler allocates a user process a slice of time (about 0.1 second) and then system clock interrupts. This entails storage of the currently running process status and selecting another runnable process to execute. This switching is done in kernel mode.

2. Services are provided by kernel by switching to the kernel mode. So if a user program needs a service (such as print service, or access to another file for some data) the operation switches to the kernel mode.

3. Suppose a user process had sought a data and the peripheral is now ready to provide the data, then the process interrupts. The hardware interrupts are handled by switching to the kernel mode.

Context Switching
(Kernel mode and User mode)

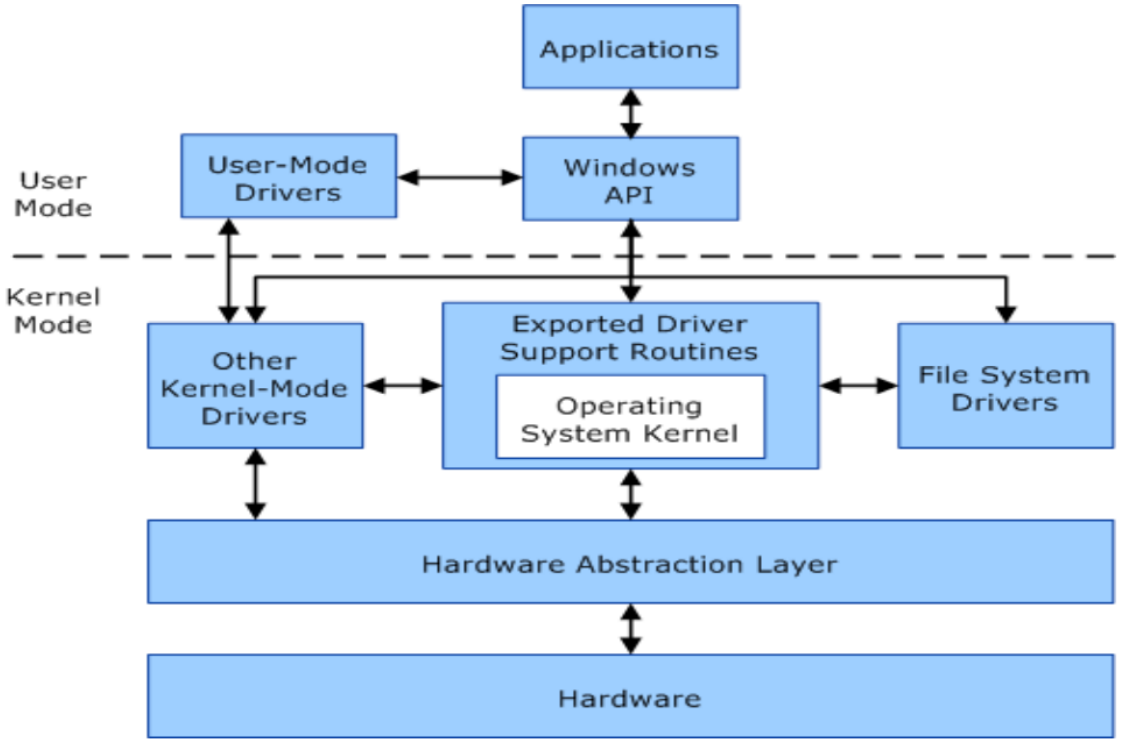


Fig: Diagram illustrating communication between user-mode and kernel-mode components.